

Keywords: vision-language-action models • robot learning
• 3D pointmaps • viewpoint generalization

See like a Robot: Robot-Centric Pointmaps for Vision-Language-Action Models

Replacing Camera-Frame RGB Images with Robot-Frame 3D Pointmaps Closes the Viewpoint Generalization Gap in Large-Scale Robot Learning Datasets

By Byungkun Lee, Dongyoon Hwang, Dongjin Kim, Hojoon Lee, Minho Park, Jaegul Choo (KAIST AI; Holiday Robotics)

arXiv:2607.11498 • July 13, 2026

Vision-language-action (VLA) models predict robot actions defined in the robot’s own 3D body frame but observe the world through camera images defined in the camera’s frame—a mismatch that is benign under fixed cameras but harmful when training datasets aggregate demonstrations from diverse camera setups. Researchers at KAIST AI and Holiday Robotics propose **robot-centric pointmaps**: images whose pixels store the 3D position of scene points in the robot frame, dropping straight into existing VLA architectures with a single channel-wise concatenation. Cross-camera evaluation improves by 20 percentage points over RGB-only baselines.

AT A GLANCE

Problem: VLA models observe scenes in the camera frame but act in the robot frame; diverse camera setups in large datasets make this frame mismatch a primary generalization barrier.

Why it matters: Large-scale robot learning requires diverse data from many camera rigs; solving viewpoint generalization unlocks the full benefit of aggregated datasets.

Method: Pointmaps— $H \times W$ images with pixel values equal to the 3D robot-frame position—are concatenated with RGB and passed to an existing VLA with one adapted input layer.

Result: Cross-camera success 61.4% vs. 41.2% for RGB only on Open X-Embodiment; +20–30pp on real-robot tasks.

Evidence: Camera-frame 3D gives only 51.3% cross-camera success, confirming the coordinate system (not just 3D info) is the key factor.

Scaling robot learning with broad, diverse datasets—analogueous to what large language models did with internet text—is a central goal in robotics. Datasets like Open X-Embodiment aggregate millions of demonstrations from dozens of robot platforms and camera setups. But unlike text, where position in the document is largely irrelevant to the downstream task, in robot learning the camera viewpoint fundamentally changes the appearance of the same scene.

A VLA trained on overhead camera demonstrations cannot directly generalize to wrist-camera deployment because the pixel-to-action mapping changes completely. If the policy could observe the scene in the robot’s own coordinate frame, viewpoint would become irrelevant: the spatial relationship between the robot and the object is the same regardless of which camera captured it.

Why Existing Approaches Fall Short

RGB-only VLAs: Learn to infer spatial relationships from monocular cues—perspective foreshortening, shadows, texture gradients. These cues are camera-dependent and do not generalize across camera setups without explicit camera-pose conditioning.

Camera-frame depth augmentation: Providing depth in the camera frame adds metric distance information but not robot-frame position. The mapping from camera-frame depth to robot-frame coordinates still requires extrinsic calibration knowledge—which may differ across demonstrations in a diverse dataset.

Camera-pose conditioning: Explicitly providing camera extrinsics as input to the policy has been explored, but requires accurate calibration at deployment and adds architectural complexity. Policies that memorize pose→action mappings may overfit to training camera configurations.

Core Mechanism

A **robot-centric pointmap** $P \in \mathbb{R}^{H \times W \times 3}$ is an image where each pixel (u, v) stores the 3D coordinate (X, Y, Z) of the corresponding scene point in the robot’s body frame. The pointmap is constructed at dataset-collection time:

1. Backproject each pixel to 3D in the camera frame using depth $d_{u,v}$ and intrinsic matrix K .
2. Transform to the robot frame using the known extrinsic $T_{\text{cam}} \rightarrow T_{\text{robot}}$.
3. Store the result as the pixel value.

The policy receives $[I_{\text{RGB}}, P] \in \mathbb{R}^{H \times W \times 6}$ as input. The visual encoder’s first layer is adapted to accept 6 channels (the 3 new channels initialized to zero weights); all other weights are kept from pretrained initialization.

The Big Picture

Technical Formulation

For pixel (u, v) with depth $d_{u,v}$, the backprojection is:

$$\mathbf{p}_{\text{cam}} = d_{u,v} K^{-1} \begin{bmatrix} u \\ v \\ 1 \end{bmatrix}$$

Transformation to the robot frame via rigid-body transform $T = (R, \mathbf{t})$:

$$\mathbf{p}_{\text{robot}} = R \mathbf{p}_{\text{cam}} + \mathbf{t}$$

The pointmap entry is $P_{u,v} = \mathbf{p}_{\text{robot}}$. This computation runs at over 60 Hz on standard GPU hardware.

For datasets where depth sensors are unavailable, the authors use Depth Anything V2 (monocular estimator) followed by scale-and-shift alignment using known anchor points (e.g., robot end-effector position). Monocular pointmaps lag behind sensor-depth pointmaps by only 4.2pp on cross-camera evaluation, showing practical viability without depth hardware.

Learning or Inference Procedure

Training follows the standard VLA pipeline—cross-entropy on action tokens given visual and language input—with the single change of replacing RGB-only input with 6-channel RGB+pointmap input. No changes to the loss, optimizer, tokenizer, or action discretization are required.

At deployment, the robot’s camera calibration (intrinsics and extrinsics relative to its body) is used to compute pointmaps in real time. The model is never given raw camera-frame depth or explicit extrinsics as an input token; the coordinate transformation is baked into the pointmap pixels.

What the Guarantee Says

If the camera-to-robot transform T is known exactly and the depth estimate $d_{u,v}$ is accurate, the pointmap provides exact robot-frame coordinates. Under this assumption, two demonstrations of the same robot-object configuration from different cameras produce identical pointmaps—guaranteeing that the policy input is camera-invariant.

In practice, depth estimation noise and calibration error introduce per-pixel errors in the pointmap. The empirical results show that the policy is robust to these errors up to sensor-depth accuracy levels.

Experimental Findings

Open X-Embodiment cross-camera (task success ↑):

Model	Same-cam.	Cross-cam.
π_0 (RGB only)	74.3	41.2
π_0 + depth (cam. frame)	77.1	46.8
π_0 + pointmap (robot frame)	80.6	61.4

Real-robot manipulation (50 trials each):

Task	RGB only	+ Pointmap
Pick (varied height)	42%	72%
Place (varied surface)	58%	84%
Reach past obstacle	34%	64%

The largest absolute gains appear in tasks requiring precise distance judgment—picking from varying heights and navigating reach obstacles—consistent with the hypothesis that robot-frame metric information is the primary missing signal.

Ablations and Interpretation

Camera-frame vs. robot-frame 3D: Using depth in the camera frame yields cross-camera success of 46.8% vs. 61.4% for robot-frame pointmaps. 3D information helps only when it is in the robot’s native coordinate system.

Monocular vs. sensor depth: Sensor depth yields 61.4% vs. 57.2% for monocular depth, a 4.2pp gap. Monocular estimation is a viable substitute in deployment environments where depth sensors are unavailable.

Channel fusion vs. early fusion: Channel concatenation (6-channel input) outperforms early-layer feature fusion (separate RGB and 3D branches merged at layer 3) by 2.8pp. Direct concatenation preserves the ability to use pretrained visual features on RGB while accessing 3D separately.

Effect of dataset diversity: Training with 10 camera configurations (vs. 2) improves cross-camera accuracy by 8.6pp for RGB-only but only 2.1pp for pointmap-based models. Pointmaps substantially reduce the benefit of camera-diverse training data, confirming that robot-frame grounding eliminates most of the viewpoint variance.

Reference

Byungkun Lee, Dongyoon Hwang, Dongjin Kim, Hojoon Lee, Minho Park, Jaegul Choo. **See like a Robot: Robot-Centric Pointmaps for Vision-Language-Action Models.** arXiv:2607.11498, July 13, 2026. <https://arxiv.org/abs/2607.11498>